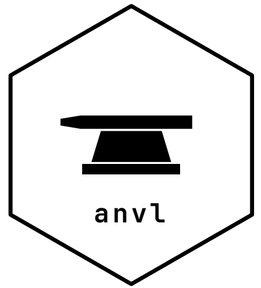
anvl: Accelerated Array Computing and Code Transformations

R+AI 2026

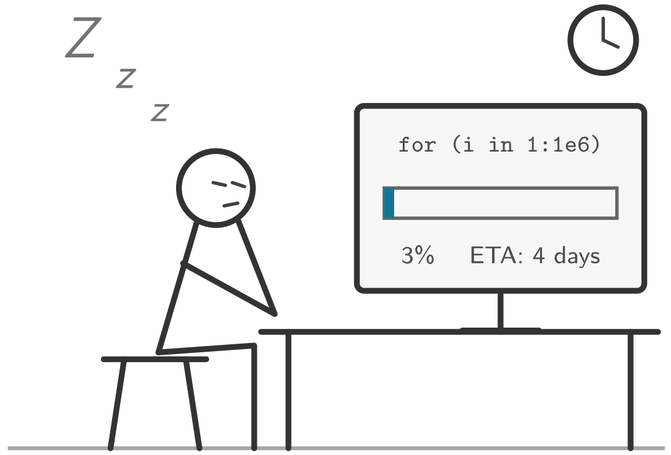
Sebastian Fischer, Louis Aslett, Daniel Falbel, Tomasz Kalinowski,
Nikolai German, Hugh Graham, Troy Hernandez, Martin Binder, Bernd Bischl
and contributors

November 11, 2026

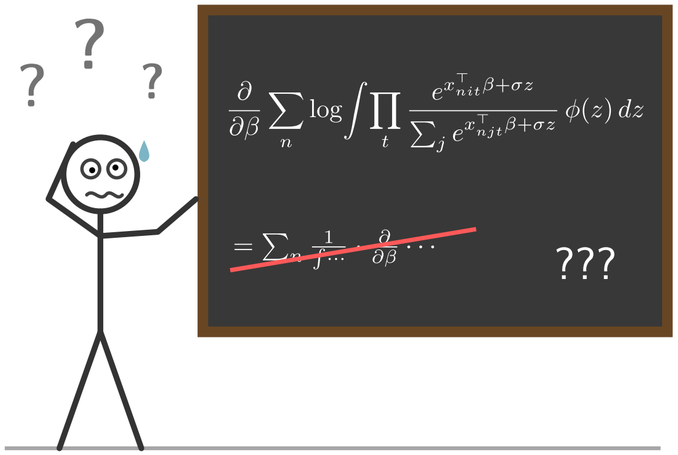


The problem

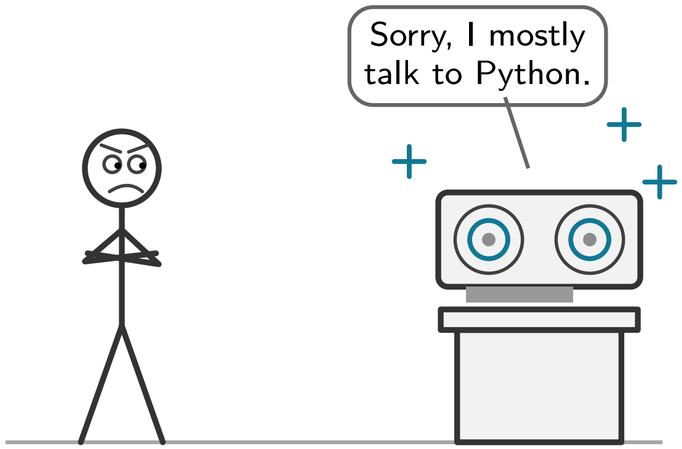
When you are implementing numeric algorithms in R, there can be three main challenges:



Speed. Iterative algorithms mean loops, and loops in R are slow.



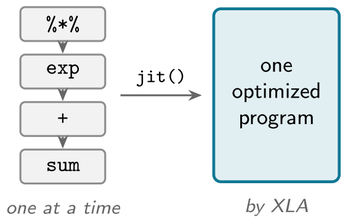
Gradients. Maximum likelihood, standard errors, HMC, variational inference and Laplace approximations all need them.

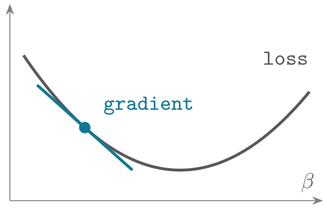


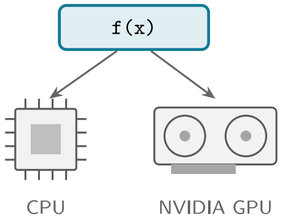
GPU. Certain algorithms are orders of magnitude faster on a GPU.

{anvl} to the rescue

You write plain R functions over arrays. {anvl} does three things with them:







Compiles. Just-in-time (JIT) compiles the whole function into one optimized program, via XLA, the compiler that also powers JAX and TensorFlow.

Differentiates. Returns the derivative of a function as another R function.

Hardware agnostic. The same code runs on CPU or NVIDIA GPU.

The design is inspired by JAX, Python's library for accelerated array computing and code transformations.

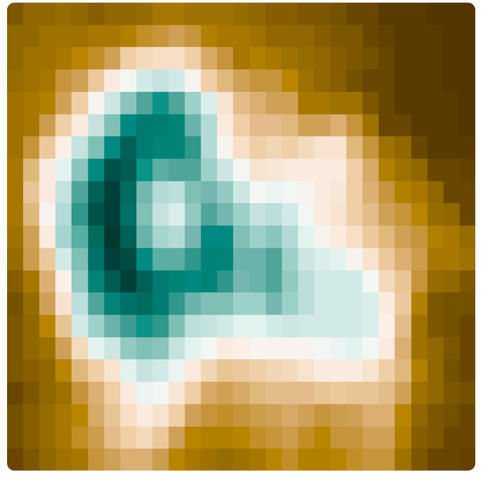
Options in R

	Compiles	Autodiff	GPU	Native R
{Rcpp}	✓	✗	✗	✗
{torch}	partly*	✓	✓	✓
JAX & co. via {reticulate}	✓	✓	✓	✗
Stan	✓	✓	partly**	✗
{quickr}	✓	✗	✗	✓
{ast2ast}, {TMB}/{RTMB}, {nimble}	✓	✓	✗	mostly
{anvl}	✓	✓	✓	✓

* {torch} can compile code via TorchScript, but TorchScript is no longer developed and does not cover all the operations we care about.
** Stan runs only some functions on the GPU, via OpenCL, such as Cholesky decompositions and GLM likelihoods; the rest of the model stays on the CPU.

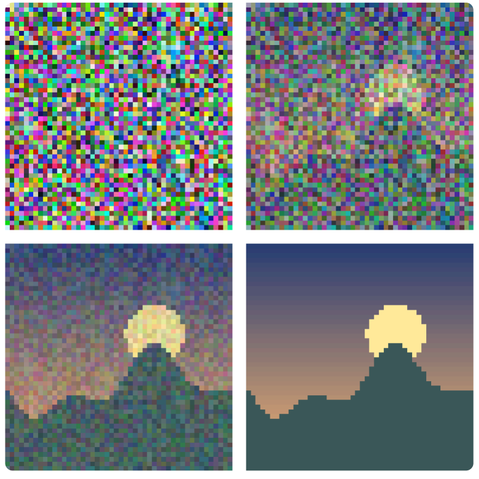
{anvl} has the distinguishing feature set: compilation, autodiff, and CPU + GPU support, natively in R.

Already building on {anvl}



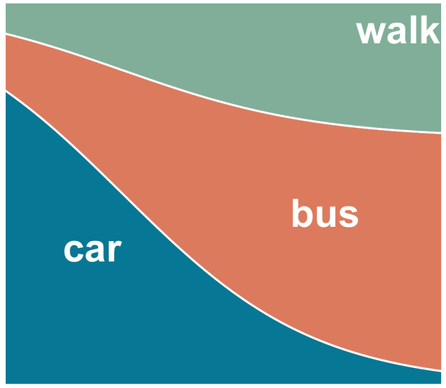
TODAY

Spatial modeling: {garry}[1], a raster engine for remote sensing that compiles its numerical kernels with {anvl} (Hugh Graham)



TODAY

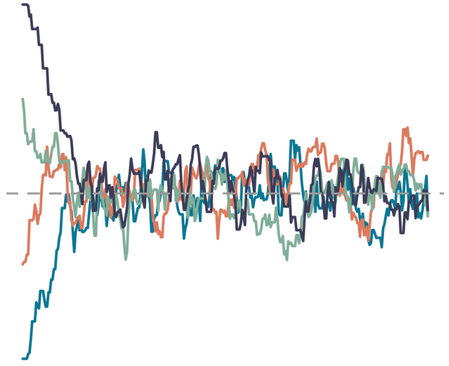
Diffusion models: Troy Hernandez (cornball.ai) is porting FLUX image-generation models to {anvl}; a FLUX.2 transformer block already runs 1.3× faster than {torch}[2] on an NVIDIA GPU at matched `f32` precision



cost of driving

IN PROGRESS

Choice modeling: the {apollo} authors are evaluating {anvl} as a backend; for a mixed logit model, up to 21× faster than pure R, on the CPU alone



iteration

IN PROGRESS

Bayesian inference: MCMC, and statistical distribution functions for {anvl} (Louis Aslett)

[1] github.com/belian-earth/garry [2] github.com/r-xla/anvl/issues/379

An R-like API

The {anvl} API

	Examples	What it is
Arrays	`nv_array()`, `nv_fill()`, `nv_seq()`	data on CPU or GPU, with a data type and a shape
Array API	`+`, `%*%`, `sum()`, `nv_solve()`, `nv_while()`	~140 functions: R generics plus `nv_*()` functions
Transformations	`jit()`, `gradient()`	take an R function, return a compiled or differentiated one

Arrays that feel like R

```
1 x <- nv_array(matrix(1:6, nrow = 2),
2               dtype = "f32", device = "cpu")
3 x
```

```
AnvlArray
 1 3 5
 2 4 6
[ CPUf32{2,3} ]
```

```
1 x_gpu <- nv_array(matrix(1:6, nrow = 2),
2                   dtype = "f32", device = "cuda")
3 x_gpu
```

```
AnvlArray
 1 3 5
 2 4 6
[ CUDAf32{2,3} ]
```

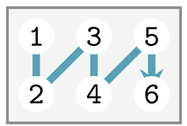
```
1 sum(x * 2)
```

```
AnvlArray
 42
[ CPUf32{} ]
```

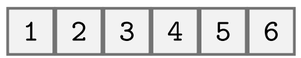
- About **100 R generics** work on arrays. Where {anvl} needs its own functions, names and arguments follow base R: `nv_pmax()`, `nv_ifelse(test, yes, no)`, `nv_seq(from, to, by)`.
- {anvl} deviates from R's semantics only for good reasons, such as consistency: a scalar index drops an axis, an array index never does, whatever its length.

Column-major, like R

R and {anvl}: column by column



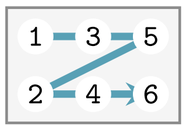
in memory



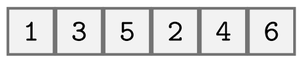

```
nv_flatten(x)
```

```
AnvlArray
 1
 2
 3
 4
 5
 6
[ CPUf32{6} ]
```

{torch}: row by row



in memory




```
xt <- torch::torch_tensor(as_array(x))
torch::torch_flatten(xt)
```

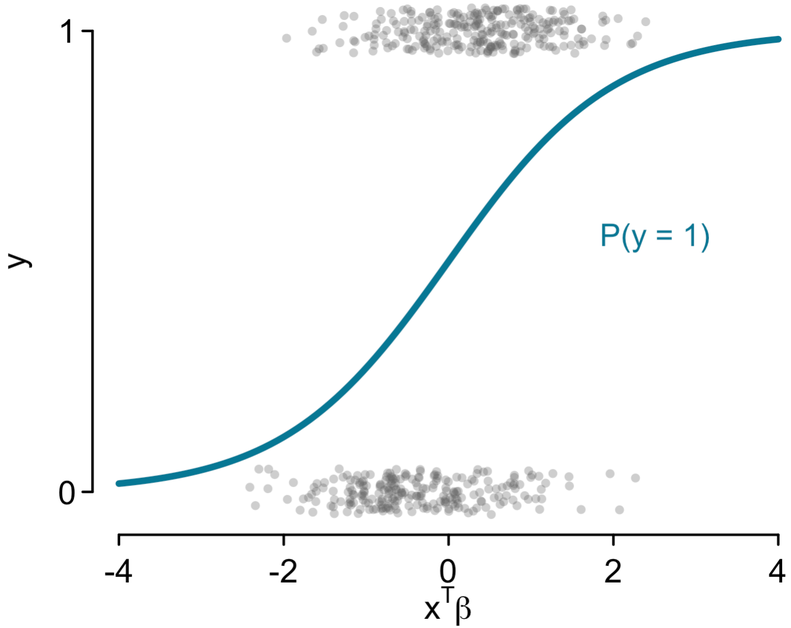
```
torch_tensor
 1
 3
 5
 2
 4
 6
[ CPUFloatType{6} ]
```

A simple example: logistic regression via gradient descent

The loss is plain R

```
1 nll <- function(beta, X, y) {
2   eta <- X %*% beta
3   -nv_mean(y * eta - nv_log1p(nv_exp(eta)))
4 }
5
6 X_nv  <- nv_array(X)          # 10,000 x 10
7 y_nv  <- nv_array(y)          # 10,000 x 1
8 beta0 <- nv_array(matrix(0, 10, 1))
9 nll(beta0, X_nv, y_nv)
```

```
AnvlArray
 0.6931
[ CPUf32{} ]
```

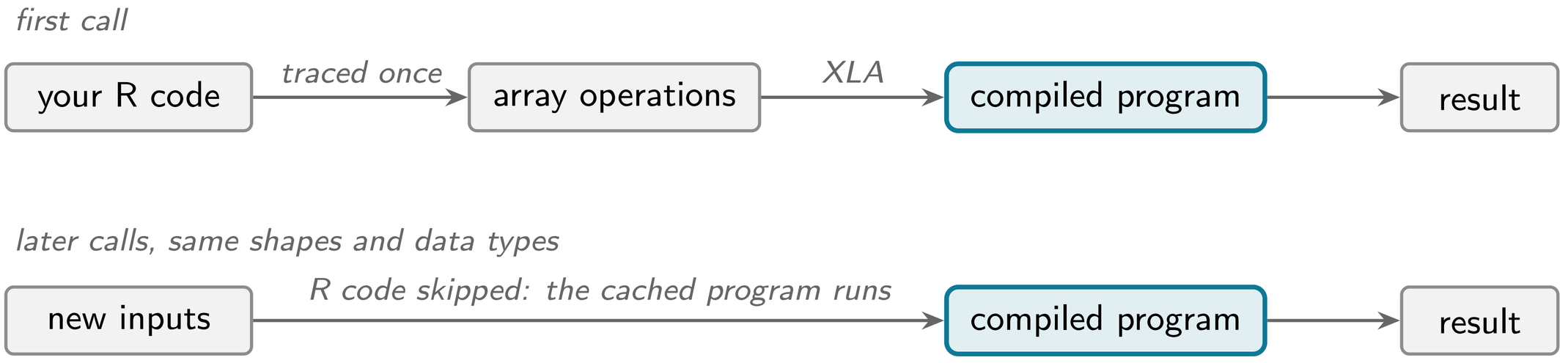


Plain R: generic operators like `%*%` and `*` work on arrays, next to `nv_*()` functions such as `nv_log1p()`.

`jit()`: compile once, run fast

```
1 nll_jit <- jit(nll)
2 nll_jit(beta0, X_nv, y_nv)
```

```
AnvlArray
 0.6931
[ CPUf32{} ]
```

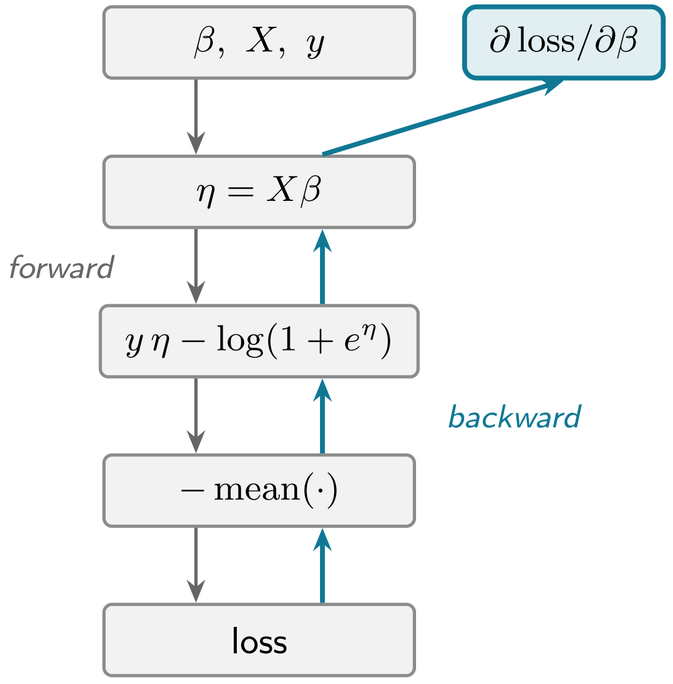


Later calls with the same shapes and data types **skip your R code entirely**.

Gradients for free

```
1 grad_nll <- jit(gradient(nll, wrt = "beta"))
2 g <- grad_nll(beta0, X_nv, y_nv)$beta
3 g
```

```
AnvlArray
 -0.0882
 -0.0476
  0.0539
  0.0436
  0.0240
 -0.0609
  0.1058
  0.0035
  0.0480
 -0.0860
[ CPUf32{10,1} ]
```



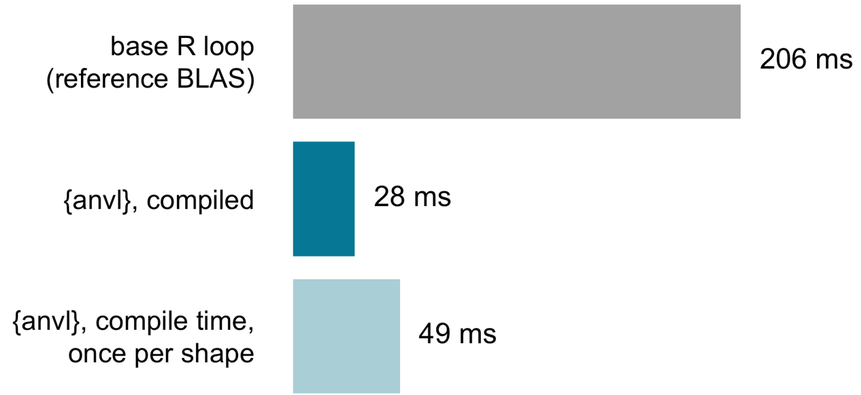
{anvl} uses **reverse mode**, which is efficient when the input size is much larger than the output size.

The whole fit, one program

```
fit <- jit(function(beta, X, y, lr, n_steps) {
  nv_while(
    init = list(beta = beta, i = 0L),
    cond = \(beta, i) i < n_steps,
    body = \(beta, i) {
      g <- grad_nll(beta, X, y)$beta
      list(beta = beta - lr * g, i = i + 1L)
    }
  )$beta
})

beta_hat <- fit(beta0, X_nv, y_nv,
                lr = 1, n_steps = 500L)
beta_glm <- coef(glm(y ~ X - 1, family = binomial))
max(abs(as_array(beta_hat) - beta_glm))
```

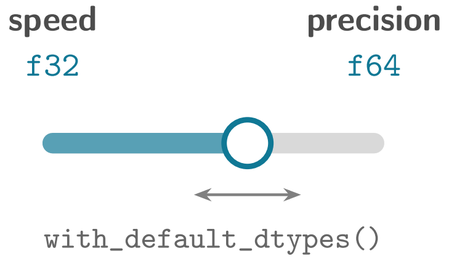
```
[1] 1.214172e-07
```



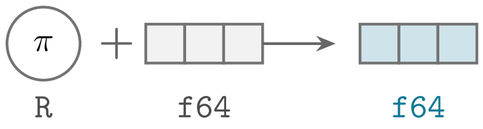
Median per call; 500 steps, n = 10,000, p = 10, `f64`; on an M1 Pro Mac.

`nv_while()` puts the loop **inside** the compiled program: one R call runs all 500 steps. `n_steps` is an ordinary input, so another number of steps reuses the same program.

Precision first

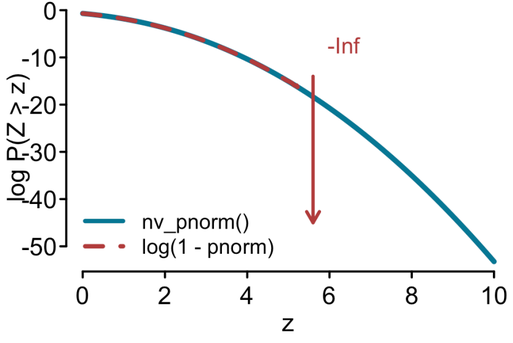


Configurable default float and integer data types: switch easily between speed (`f32`) and precision (`f64`)

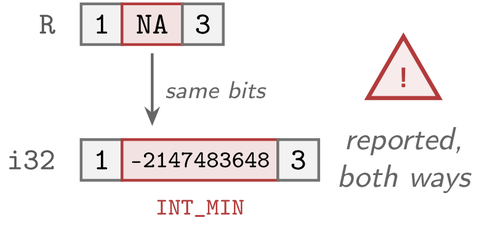


full precision, like base R

Carefully designed type promotion rules: R values never lose precision: they take the data type of the array they meet, e.g. `pi` with an `f64` array



High-precision distribution functions, carefully benchmarked and accurate far into the tails*



Careful `NA` handling: R stores `NA_integer_` as `INT_MIN`; {anvl} reports it when data enters, and when such a value comes back to R

* Still less precise than base R, the gold standard our implementations are benchmarked against.

Summary of noteworthy features

No CUDA toolkit: `install_anvl()` installs the CUDA libraries as an R package; an NVIDIA driver is all you need

Serializable arrays: `saveRDS()` works, unlike for most objects that wrap external pointers; `nv_save()` writes safetensors

Memory R can see: R's garbage collector accounts for CPU results; on a GPU, running out of memory triggers `gc()` and a retry

All CPU cores, with nothing to configure

Asynchronous execution: R keeps going while the device computes

Nested lists of arrays as inputs and outputs, also for `gradient()`

An alternative backend: experimental {quickr}, which transpiles to Fortran

Mostly written in R: easy to extend and contribute to, without a low-level language

Careful error messages: they say what went wrong and how to fix it

Why you should adopt {anvl} for your modeling package

A high degree of customization: instead of picking from options you hard-coded, your users write parts of the model as R functions. Here: the utility of each alternative in a discrete choice model.

In your package

```r
# Multinomial logit: log-likelihood
mnl_loglik <- jit(function(beta, data, utilities) {
  V <- lapply(utilities, \(u) u(beta, data))
  choice_loglik(V, data$choice)
}, static = "utilities")
# ... and the helper choice_loglik()
```

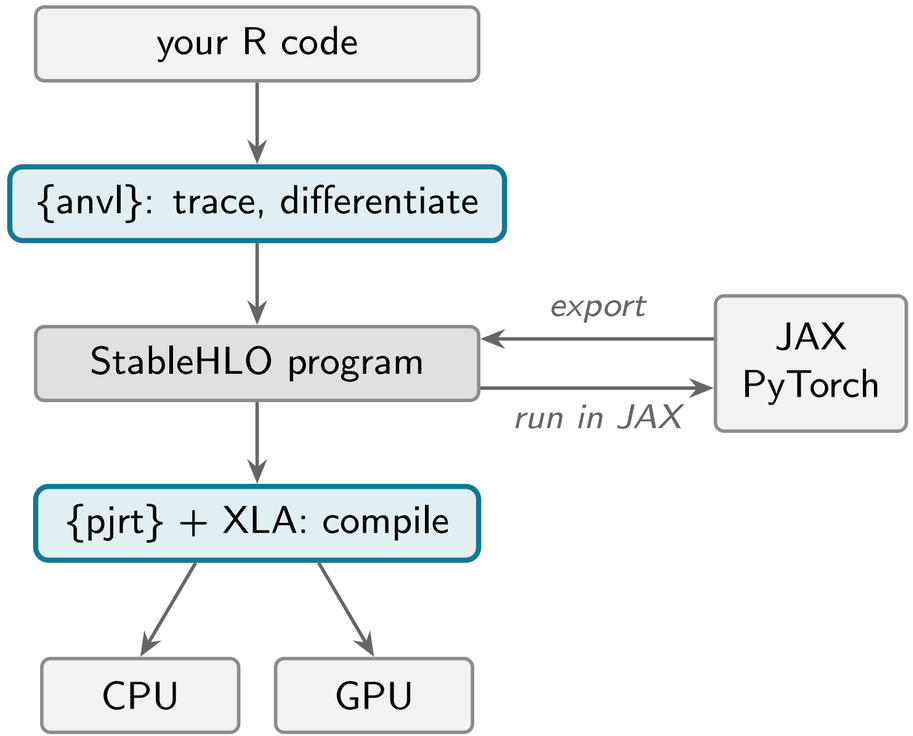
Your users

```r
utilities <- list(
  car  = \(b, d) b$asc_car + b$time * d$time_car + b$cost * d$cost_car,
  bus  = \(b, d) b$asc_bus + b$time * d$time_bus + b$cost * d$cost_bus,
  walk = \(b, d) b$time * d$time_walk
)
mnl_loglik(beta, data, utilities)
```

- One compiled program from your package code and the user's utilities: as fast as if you had written them yourself, and differentiable end to end, for maximum likelihood estimation
- No C++ in your package, nothing to compile for your users, and the same code runs on a GPU

Built on industry standards, extensible in R

Built on OpenXLA



StableHLO is a portable format: models exported from JAX, or from PyTorch via torchax, run in R without Python, and most {anvl} programs run in JAX.

Extensible in R

- Missing a function? Write it from `nv_*()` functions, in R: compiled, fused and differentiable, like a built-in (Adding an API Function[1])
- Need more? Call your own C++ or CUDA code (Adding Custom Calls[2])

[1] r-xla.github.io/anvl/articles/extending_api.html [2] r-xla.github.io/pjrt/articles/ffi.html

Wrapping up

Is {anvl} for you?

Good fit

- ✓ Iterative algorithms: optimizers, samplers, simulations
- ✓ Anything that needs gradients
- ✓ Large arrays, linear algebra, many chains
- ✓ Code you want to run on an NVIDIA GPU
- ✓ Packages that need gradients and compiled code underneath

Better elsewhere

- ✗ Small one-off computations: base R
- ✗ Data wrangling, strings, filtering: base R and friends
- ✗ GPUs other than NVIDIA: not supported yet
- ✗ Sparse matrices: not supported

Limitations

Static shapes

Each new combination of input shapes compiles a new program, and no output's shape can depend on the data. Padding inputs to a few fixed sizes helps: Static Shape Restriction[1], padding inputs[2].

	Expensive	Cheap
Many input shapes	✓ Good fit, with padding	– Not needed
Few input shapes	✓ Good fit	– Not needed

`jit()` takes some getting used to

Your R code runs once, while tracing, so branching and looping on array values need `nv_if()` and `nv_while()`, and getting this right takes some care: JIT Deep Dive[3].

Some more peculiarities to get used to: Gotchas[4].

[1] r-xla.github.io/anvl/articles/static_shapes.html [2] r-xla.github.io/anvl/articles/efficiency.html
[3] r-xla.github.io/anvl/articles/jit.html [4] r-xla.github.io/anvl/articles/gotchas.html

Get started

{anvl} is not on CRAN yet, so install it from r-universe:

```
install.packages(
  "anvl",
  repos = c("https://r-xla.r-universe.dev", getOption("repos"))
)
anvl::install_anvl()
```

For an NVIDIA GPU, on Linux or on Windows via WSL2, **a compatible driver is all you need**: no CUDA toolkit to install. On macOS, {anvl} runs on the CPU. Prebuilt Docker images: `ghcr.io/r-xla/anvl-cpu` and `ghcr.io/r-xla/anvl-cuda`.

The package website, r-xla.github.io/anvl, has a Get Started article[1] and worked examples: a Gaussian process, a Metropolis-Hastings sampler.

[1] r-xla.github.io/anvl/articles/anvl.html

Where this is going

Current work

- **Distribution functions:** more densities, distribution functions and quantiles, accurate in the tails
- **Auto-vectorization:** write a function for a single item, map it across a batch
- **CRAN release:** in progress; for now, {anvl} is on r-universe

Potential next steps

- **Forward-mode AD:** combined with reverse mode, efficient Hessians, e.g. for Laplace approximations
- Easier custom C++ and CUDA kernels

Possibly, via the IREE compiler

- No static-shape restriction
- More hardware: Apple GPUs (Metal), maybe WebAssembly

There's lots to do: contributions welcome! And feedback is always very helpful to steer {anvl} towards what benefits the R community.

Thank you

Try it on your own algorithm, and tell us where it breaks.

- Package: github.com/r-xla/anvl
- Docs: r-xla.github.io/anvl

Acknowledgments

This work is supported by **MaRDI**, the Mathematical Research Data Initiative, an NFDI project funded by the Deutsche Forschungsgemeinschaft (DFG), project number 460135501.

